1 Introduction

Tomasulo Algorithm. Tomasulo's algorithm uses reservation stations, register renaming and result forwarding to execute ready instructions out of order. The reorder buffer keeps architectural updates in program order.

This assignment extends the supplied Python implementation to issue 1-4 instructions per cycle. The ISSUE, EX, MEM, WB and COMMIT module structure is retained.

• Added a configurable issue width, dependency-safe renaming within each bundle and stalls when the next instruction has no free resources.

• Tested both sample-report inputs and seven additional programs at all four widths, with timing and final-state checks.

What Works

• In-order multiple issue, out-of-order execution, precise retirement and all ten specified opcodes.

• Pipelined FP units, configurable hardware, branch prediction/recovery, store forwarding and memory replay.

Limitations

• Writeback and retirement remain one instruction per cycle. Memory has one unpipelined port; extra load/store units add address calculators.

• Python arithmetic does not model single-precision rounding or integer overflow. Caches, interrupts and general exception handlers are outside scope.

Ali Alfageeh | October 15, 2026. The sections and result figures follow the supplied sample; code and timings describe this implementation.

2 Code Changes

The changes below compare excerpts from the supplied starter with the submitted code. The starter's stage organization is kept, but state is now carried by the Simulator object.

(1) Multiple issue in issue.py. The input parser accepts issue widths 1-4, with a default of 1. Issue stops at the first full ROB or required queue.

Initial Version

issue.py, supplied starter, lines 171-179 (excerpt).

```
def issue(cycle, PC, instructions, ROB, size_ROB,
         rs_int_adder,
         rs_fp_adder,
         rs_fp_multi,
         ld_sd_queue, size_ld_sd_queue,
         rat_int, rat_fp):
    # fetch 1 instruction
    ins = instructions[PC.PC]
    op = ins.split(' ')[0]
```

My Version

```
def issue(cpu):
    if cpu.cycle < cpu.fetch_after:
        return
    for _ in range(cpu.config.issue_width):
        if cpu.pc == len(cpu.program.instructions):
            return
        instruction = cpu.program.instructions[cpu.pc]
        unit = instruction.unit
        queue = cpu.lsq if unit == "address" else cpu.stations[unit]
        if (len(cpu.rob) == cpu.config.rob_entries or
                len(queue) == getattr(cpu.config, unit).stations):
            cpu.issue_stalls += 1
            return  # A younger instruction cannot bypass a blocked instruction.
```

A blocked instruction stops the bundle, so younger work cannot pass it. Instructions already issued during that cycle remain issued.

2.1 Source operands and renaming

The starter stores either a value or a tag in its RAT arrays. The revised version keeps the architectural registers separate and looks up the exact producer tag.

Initial Version

issue.py, supplied starter, lines 48-65 (excerpt).

```
# reg 1
register = ins.split(' ')[2] # register name
if register[0] == 'F':
    if (type(rat_fp[int(register[1:])]) == str):
        entry.tag_1st = rat_fp[int(register[1:])]
        entry.valid_1st = 0
    else:
        entry.value_1st = rat_fp[int(register[1:])]
        entry.valid_1st = 1
elif register[0] == 'R':
    if (type(rat_int[int(register[1:])]) == str):
        entry.tag_1st = rat_int[int(register[1:])]
        entry.valid_1st = 0
    else:
        entry.value_1st = rat_int[int(register[1:])]
        entry.valid_1st = 1
else:
    pass
```

My Version

```
def source(cpu, name):
    tag = cpu.rat.get(name)
    if tag is None:
        return Operand(cpu.registers[name])
    producer = next(entry for entry in cpu.rob if entry.tag == tag)
    if producer.wb is not None:
        return Operand(producer.result, ready_at=producer.wb)
    return Operand(tag=tag)
```

Source operands are captured before the destination updates the RAT. This lets later instructions in the same bundle depend on earlier ones. Each instruction gets a new tag, so a reused register name cannot identify the wrong producer.

2.2 Writeback

The starter writes the broadcast value directly into the RAT entry for the destination register. If a younger instruction has renamed that register, an older completion must not erase its mapping.

Initial Version

wb.py, supplied starter, lines 26-31 (excerpt).

```
# rat_int
if reg_tag[0] == 'R':
    rat_int[int(reg_tag[1:])] = cdb.value
# rat_fp
if reg_tag[0] == 'F':
    rat_fp[int(reg_tag[1:])] = cdb.value
```

My Version

```
def wb(cpu):
    candidates = [entry for entry in cpu.rob
                  if entry.complete is not None and entry.complete < cpu.cycle
                  and entry.wb is None and entry.error is None
                  and entry.instruction.dest is not None]
    if not candidates:
        return
    entry = min(candidates, key=lambda item: item.tag)
    entry.wb = cpu.cycle
    for consumer in cpu.rob:
        for operand in consumer.operands:
            if operand.tag == entry.tag:
                operand.tag = None
                operand.value = entry.result
                operand.ready_at = cpu.cycle
```

Only consumers waiting for the selected dynamic tag receive the result. The single CDB broadcasts at most one eligible result per cycle, as specified in the handout. Issue width does not increase the writeback width.

2.3 Commit

Only the head of the ROB may update architectural state. The revised retirement helper removes a RAT entry only if it still names the retiring instruction.

Initial Version

commit.py, supplied starter, lines 27-38 (excerpt).

```
def commit(ROB, reg_int, reg_fp, cycle, instructions):
    if len(ROB)>0:
        if instructions[ROB[0].PC].split(' ')[0]=='Bne':
            if len(ROB[0].exe)!=0:
                entry = ROB.popleft()
                print_ROB(entry, instructions)
        if (len(ROB)>0)&(instructions[ROB[0].PC].split(' ')[0]!='Bne'):
            if len(ROB[0].cdb)!=0: # broadcasted instructions
                ROB[0].commit.append(cycle+1)
                entry = ROB.popleft()
                modify_arch_reg(entry, reg_int, reg_fp)
                print_ROB(entry, instructions)
```

My Version

```
def retire(cpu, entry):
    entry.commit = cpu.cycle
    cpu.rob.pop(0)
    destination = entry.instruction.dest
    if destination and destination != "R0":
        cpu.registers[destination] = entry.result
        if cpu.rat.get(destination) == entry.tag:
            del cpu.rat[destination]
```

The commit stage checks that a register-producing instruction wrote back in an earlier cycle. A store starts its memory transfer only at the ROB head, with both address and data ready; memory changes on its final transfer cycle. At most one instruction retires per cycle.

This prevents a wrong-path store from updating memory before an older unresolved branch.

2.4 Execution cycle accounting

In the starter, fu_exe increments an operation's counter whenever it is called. The scheduler can call it again after accepting waiting work in the same cycle. The revised code records an explicit completion cycle instead.

Initial Version

exe.py, supplied starter, lines 50-53 (excerpt).

```
def fu_exe(fu, fu_results, ROB, time_fu, cycle, PC):
    if len(fu)!=0:
        for element in fu:
            element.cycle += 1
```

My Version

exe.py, scheduling and completion excerpts:

```
entry.ex_start = cpu.cycle
cpu.executing.append(Execution(entry, cpu.cycle + config.latency - 1))

for task in sorted(list(cpu.executing), key=lambda item: item.entry.tag):
    if task.entry.squashed is None and task.end == cpu.cycle:
        cpu.executing.remove(task)
        finish(cpu, task.entry)
```

An operation that begins in cycle n with latency L finishes in n+L-1. A one-cycle operation therefore begins and ends in the same EX cycle. It cannot start in its issue cycle or write back in its completion cycle.

2.5 Selecting ready instructions

The starter scans reservation stations for ready operands. The revised scheduler also checks cycle timestamps and available functional units.

Initial Version

exe.py, supplied starter, lines 15-23 (excerpt).

```
def check_valid_ins_in_rs(rs):
    index = -1
    if len(rs)!=0:
        for i in range(len(rs)):
            if (rs[i].valid_1st==1)&(rs[i].valid_2nd==1)&(rs[i].busy==1):
                index = i
                break
    return index
```

My Version

```
for entry in list(queue):
    # Store address generation does not wait for store data.
    operands = entry.operands[:1] if unit == "address" else entry.operands
    if (slots <= 0 or entry.ex_start is not None or entry.issue >= cpu.cycle
            or not all(operand.ready(cpu.cycle) for operand in operands)):
        continue
```

Entries remain in issue order within each queue. The scheduler selects the oldest ready work for that unit, while a blocked older instruction may wait. Execution can therefore be out of order even though issue and retirement are in order.

Integer and address units remain occupied until completion. FP add and multiply units are pipelined and can accept new work each cycle, up to their configured count.

2.6 Load/store ordering

The starter's forwarding search stops at the nearest older store. The revised search selects the youngest older store at the same address, even when a nonmatching store lies between it and the load.

Initial Version

mem.py, supplied starter, lines 19-28 (excerpt).

```
def forward_check_from_sd(ld_sd_queue, index):
    data = []
    for i in reversed(range(index)):
        if (ld_sd_queue[i].op=='Sd'):
            if (ld_sd_queue[i].address == ld_sd_queue[index].address):
                if (type(ld_sd_queue[i].data)==int)|(type(ld_sd_queue[i].data)==float):
                    data.append(ld_sd_queue[i].data)
            break
    return data
```

My Version

```
stores = [older for older in cpu.lsq if older.tag < entry.tag
          and older.instruction.op == "sd" and older.address == entry.address]
store = stores[-1] if stores else None
if store and not store.operands[1].ready(cpu.cycle):
    continue
entry.forwarded = store is not None
value = store.operands[1].value if store else cpu.memory[entry.address]
latency = 1 if store else cpu.config.memory_latency
```

The load waits if the matching store's data is unavailable. Forwarding takes one cycle; other accesses use the configured memory latency. The memory port is single-ported and unpipelined, and a load leaves the queue when its value returns.

Loads may pass stores with unknown addresses. When an older store resolves to an address already read by a younger load, that load and all younger work are replayed. The check includes loads still in the ROB after leaving the queue.

2.7 Order of pipeline stages

The stage calls run in reverse order so a result from a later stage is not used too early by a preceding stage during the same cycle.

Initial Version

main.py call order, with arguments omitted:

```
issue(...)
exe(...)
mem(...)
wb(...)
commit(...)
```

My Version

simulator.py:

```
def step(self):
    self.cycle += 1
    commit(self)
    wb(self)
    mem(self)
    exe(self)
    issue(self)
```

Timestamps provide a second check. A consumer cannot execute in the cycle its producer broadcasts. For Addi R1 R0 7 followed by Add R2 R1 R1 at width 2, both issue in cycle 1. The first executes in 2 and broadcasts in 3; the second executes in 4 and broadcasts in 5. They commit in cycles 4 and 6, leaving R2=14.

2.8 Branch recovery

The starter waits for its Bne handling to make the PC valid again. The revised version predicts during issue and resolves both Beq and Bne in EX.

Initial Version

exe.py, supplied starter, lines 71-75 (excerpt).

```
elif (fu[0].op=='Bne'):
    fu_results.pop()
    if fu[0].value1 == fu[0].value2:
        PC.PC += 1
        PC.valid = 1
```

My Version

```
if instruction.branch:
    taken = (values[0] == values[1]) == (instruction.op == "beq")
    target = entry.pc + 1 + instruction.immediate
    actual = target if taken else entry.pc + 1
    cpu.btb[entry.pc % 8] = Prediction(entry.pc, target, taken)
    cpu.branches += 1
    if actual != entry.predicted_pc:
        cpu.mispredictions += 1
        cpu.squash(entry.tag + 1, actual)
```

An eight-entry tagged BTB uses the low three bits of the word PC and a one-bit prediction. A mismatch squashes younger ROB entries, reservation stations, memory work and executing instructions. The RAT is rebuilt from surviving writers, and correct fetch resumes in cycle n+2 after resolution in n.

Branch offsets are signed words: byte target = PC + 4 + offset * 4. The eight-instruction vector loop uses -8 to return to its first instruction; the starter's legacy -12 immediate was corrected for this convention.

3 Results

The next pages use the sample report's two hardware configurations and instruction lists. Each case is followed by four separate simulation figures, one for each issue width. These are fresh results from the submitted simulator.

Benchmark	W1	W2	W3	W4
branches	25	20	20	20
dependencies	20	20	20	20
independent	20	20	20	20
late_critical_path	27	23	21	21
memory	42	41	40	40
sample_report_01	36	36	36	36
sample_report_02	47	46	46	46
structural	40	40	40	40
vector_loop	66	65	65	65

The values are simulated cycles. All 36 runs match a separate sequential interpreter in registers, memory and committed instruction sequence.

The sample report widens writeback and retirement and allows different memory concurrency. This implementation keeps one CDB, one memory port and one retirement per cycle, so its results differ. Sample case 01 takes 36 cycles at every width; case 02 takes 47 cycles at width 1 and 46 at widths 2-4.

Wider issue helps when it reaches ready work sooner. The late-critical-path program improves from 27 to 21 cycles. Dependencies, memory serialization and retirement still limit the other programs.

3.1 TestCases

3.1.1 Test Case #01

Configuration of the Architecture

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	4	3		1
FP multiplier	4	5		1
Load/store unit	4	1	6	1

ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

List of Instructions

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Input: benchmarks/sample_report_01.txt. The next four figures show issue widths 1, 2, 3 and 4.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[2]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[3]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[4]	[11, 15]	[]	[16]	[21]
Add.d F4 F2 F4	[5]	[17, 19]	[]	[20]	[22]
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[23]
Ld F1 0(R1)	[7]	[9, 9]	[10, 15]	[17]	[24]
Mult.d F2 F1 F0	[8]	[18, 22]	[]	[23]	[25]
Add.d F3 F3 F2	[9]	[24, 26]	[]	[27]	[28]
Mult.d F2 F1 F1	[10]	[19, 23]	[]	[24]	[29]
Add.d F4 F2 F4	[11]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[12]	[13, 13]	[]	[14]	[31]
Ld F1 0(R1)	[13]	[15, 15]	[16, 21]	[22]	[32]
Mult.d F2 F1 F0	[14]	[23, 27]	[]	[29]	[33]
Add.d F3 F3 F2	[16]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[17]	[24, 28]	[]	[30]	[35]
Add.d F4 F2 F4	[18]	[31, 33]	[]	[34]	[36]

Figure 1: Simulation results for Test-case #01 with Issue-width #01

Total cycles: 36. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=28, R2=32, F1=5, F2=25, F4=26, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_01_w1.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[2]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[21]
Add.d F4 F2 F4	[3]	[17, 19]	[]	[20]	[22]
Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[23]
Ld F1 0(R1)	[4]	[6, 6]	[9, 14]	[17]	[24]
Mult.d F2 F1 F0	[4]	[18, 22]	[]	[23]	[25]
Add.d F3 F3 F2	[5]	[24, 26]	[]	[27]	[28]
Mult.d F2 F1 F1	[5]	[19, 23]	[]	[24]	[29]
Add.d F4 F2 F4	[6]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[31]
Ld F1 0(R1)	[7]	[9, 9]	[15, 20]	[21]	[32]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[29]	[33]
Add.d F3 F3 F2	[16]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[30]	[35]
Add.d F4 F2 F4	[17]	[31, 33]	[]	[34]	[36]

Figure 2: Simulation results for Test-case #01 with Issue-width #02

Total cycles: 36. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=28, R2=32, F1=5, F2=25, F4=26, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_01_w2.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[21]
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[22]
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[23]
Ld F1 0(R1)	[3]	[5, 5]	[9, 14]	[17]	[24]
Mult.d F2 F1 F0	[3]	[18, 22]	[]	[23]	[25]
Add.d F3 F3 F2	[3]	[24, 26]	[]	[27]	[28]
Mult.d F2 F1 F1	[4]	[19, 23]	[]	[24]	[29]
Add.d F4 F2 F4	[4]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[4]	[5, 5]	[]	[6]	[31]
Ld F1 0(R1)	[5]	[7, 7]	[15, 20]	[21]	[32]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[29]	[33]
Add.d F3 F3 F2	[16]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[30]	[35]
Add.d F4 F2 F4	[17]	[31, 33]	[]	[34]	[36]

Figure 3: Simulation results for Test-case #01 with Issue-width #03

Total cycles: 36. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=28, R2=32, F1=5, F2=25, F4=26, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_01_w3.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]
Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]
Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]
Mult.d F2 F1 F1	[1]	[11, 15]	[]	[16]	[21]
Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[22]
Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[23]
Ld F1 0(R1)	[2]	[5, 5]	[9, 14]	[17]	[24]
Mult.d F2 F1 F0	[2]	[18, 22]	[]	[23]	[25]
Add.d F3 F3 F2	[3]	[24, 26]	[]	[27]	[28]
Mult.d F2 F1 F1	[3]	[19, 23]	[]	[24]	[29]
Add.d F4 F2 F4	[3]	[25, 27]	[]	[28]	[30]
Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[31]
Ld F1 0(R1)	[4]	[7, 7]	[15, 20]	[21]	[32]
Mult.d F2 F1 F0	[10]	[22, 26]	[]	[29]	[33]
Add.d F3 F3 F2	[16]	[30, 32]	[]	[33]	[34]
Mult.d F2 F1 F1	[16]	[23, 27]	[]	[30]	[35]
Add.d F4 F2 F4	[17]	[31, 33]	[]	[34]	[36]

Figure 4: Simulation results for Test-case #01 with Issue-width #04

Total cycles: 36. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=28, R2=32, F1=5, F2=25, F4=26, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_01_w4.txt.

3.1.2 Test Case #02

Configuration of the Architecture

	# of rs	Cycles in EX	Cycles in Mem	# of FUs
Integer adder	4	1		1
FP adder	3	3		2
FP multiplier	2	15		2
Load/store unit	4	1	5	2

ROB entries = 128
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

List of Instructions

```
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Addi R1 R1 4
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Input: benchmarks/sample_report_02.txt. The next four figures show issue widths 1, 2, 3 and 4.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[2]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[3]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[4]	[9, 23]	[]	[25]	[30]
Ld F1 0(R1)	[5]	[6, 6]	[8, 12]	[13]	[31]
Addi R1 R1 4	[6]	[7, 7]	[]	[9]	[32]
Add.d F4 F2 F4	[7]	[26, 28]	[]	[29]	[33]
Addi R1 R1 8	[8]	[10, 10]	[]	[11]	[34]
Ld F1 0(R1)	[9]	[12, 12]	[13, 17]	[18]	[35]
Mult.d F2 F1 F0	[10]	[19, 33]	[]	[34]	[36]
Add.d F3 F3 F2	[11]	[35, 37]	[]	[38]	[39]
Mult.d F2 F1 F1	[12]	[19, 33]	[]	[35]	[40]
Ld F1 0(R1)	[13]	[14, 14]	[18, 22]	[23]	[41]
Mult.d F2 F1 F0	[19]	[24, 38]	[]	[39]	[42]
Add.d F3 F3 F2	[25]	[40, 42]	[]	[43]	[44]
Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[45]
Add.d F4 F2 F4	[27]	[43, 45]	[]	[46]	[47]

Figure 5: Simulation results for Test-case #02 with Issue-width #01

Total cycles: 47. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=24, R2=32, F1=6, F2=36, F4=37, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_02_w1.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[2]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[25]	[30]
Ld F1 0(R1)	[3]	[4, 4]	[8, 12]	[13]	[31]
Addi R1 R1 4	[3]	[4, 4]	[]	[5]	[32]
Add.d F4 F2 F4	[4]	[26, 28]	[]	[29]	[33]
Addi R1 R1 8	[4]	[6, 6]	[]	[7]	[34]
Ld F1 0(R1)	[5]	[8, 8]	[13, 17]	[18]	[35]
Mult.d F2 F1 F0	[9]	[19, 33]	[]	[34]	[36]
Add.d F3 F3 F2	[9]	[35, 37]	[]	[38]	[39]
Mult.d F2 F1 F1	[10]	[19, 33]	[]	[35]	[40]
Ld F1 0(R1)	[10]	[11, 11]	[18, 22]	[23]	[41]
Mult.d F2 F1 F0	[19]	[24, 38]	[]	[39]	[42]
Add.d F3 F3 F2	[25]	[40, 42]	[]	[43]	[44]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[45]
Add.d F4 F2 F4	[26]	[42, 44]	[]	[45]	[46]

Figure 6: Simulation results for Test-case #02 with Issue-width #02

Total cycles: 46. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=24, R2=32, F1=6, F2=36, F4=37, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_02_w2.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[2]	[9, 23]	[]	[25]	[30]
Ld F1 0(R1)	[2]	[3, 3]	[8, 12]	[13]	[31]
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[32]
Add.d F4 F2 F4	[3]	[26, 28]	[]	[29]	[33]
Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[34]
Ld F1 0(R1)	[3]	[7, 7]	[13, 17]	[18]	[35]
Mult.d F2 F1 F0	[9]	[19, 33]	[]	[34]	[36]
Add.d F3 F3 F2	[9]	[35, 37]	[]	[38]	[39]
Mult.d F2 F1 F1	[9]	[19, 33]	[]	[35]	[40]
Ld F1 0(R1)	[10]	[11, 11]	[18, 22]	[23]	[41]
Mult.d F2 F1 F0	[19]	[24, 38]	[]	[39]	[42]
Add.d F3 F3 F2	[25]	[40, 42]	[]	[43]	[44]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[45]
Add.d F4 F2 F4	[26]	[42, 44]	[]	[45]	[46]

Figure 7: Simulation results for Test-case #02 with Issue-width #03

Total cycles: 46. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=24, R2=32, F1=6, F2=36, F4=37, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_02_w3.txt.

	ISSUE	EXE	MEM	WB	COMMIT
Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]
Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]
Mult.d F2 F1 F1	[1]	[9, 23]	[]	[25]	[30]
Ld F1 0(R1)	[2]	[3, 3]	[8, 12]	[13]	[31]
Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[32]
Add.d F4 F2 F4	[2]	[26, 28]	[]	[29]	[33]
Addi R1 R1 8	[2]	[5, 5]	[]	[6]	[34]
Ld F1 0(R1)	[3]	[7, 7]	[13, 17]	[18]	[35]
Mult.d F2 F1 F0	[9]	[19, 33]	[]	[34]	[36]
Add.d F3 F3 F2	[9]	[35, 37]	[]	[38]	[39]
Mult.d F2 F1 F1	[9]	[19, 33]	[]	[35]	[40]
Ld F1 0(R1)	[9]	[10, 10]	[18, 22]	[23]	[41]
Mult.d F2 F1 F0	[19]	[24, 38]	[]	[39]	[42]
Add.d F3 F3 F2	[25]	[40, 42]	[]	[43]	[44]
Mult.d F2 F1 F1	[25]	[26, 40]	[]	[41]	[45]
Add.d F4 F2 F4	[26]	[42, 44]	[]	[45]	[46]

Figure 8: Simulation results for Test-case #02 with Issue-width #04

Total cycles: 46. All 17 instructions committed. EXE and MEM intervals include both endpoints; [] means that the stage does not apply.

Final Register and Memory Values

Registers: R1=24, R2=32, F1=6, F2=36, F4=37, F20=3.

Memory: Mem[4]=3, Mem[8]=2, Mem[12]=1, Mem[24]=6, Mem[28]=5, Mem[32]=4. All omitted values are zero.

Checked against the sequential interpreter. Full output: results/sample_report_02_w4.txt.

3.2 Additional Tests

The following seven programs cover hazards and recovery that are not present in the two supplied sample streams. Each runs at widths 1, 2, 3 and 4.

Test	Purpose and selected final values
branches	Loop recovery and wrong-path stores; R2=4, R3=8; memory stays zero
dependencies	RAW/WAR/WAW and R0; F3=4, F4=7, R3=21, R0=0
independent	Mixed ready operations; F3=6, F7=9, F10=6, R4=4
late_critical_path	Late multiply/add chain; F3=6, F4=12
memory	Replay and forwarding; F3=9, F4=18; Mem[0]=9, Mem[4]=7, Mem[8]=9
structural	Two-entry ROB and small queues; F7=16, Mem[0]=8
vector_loop	Three iterations; R1=0; Mem[24]=15, Mem[28]=11, Mem[32]=7

The 20 unittest tests include hand-checked timing, tiny queues, store-data dependencies, FU pipelining, branch recovery, precise faults and CLI behavior. Forty deterministic randomized programs run at each width, for 160 differential runs.

All tests pass. No known failing case remains in this tested scope. Timing checks complement the sequential interpreter, which checks final architectural state but does not model execution cycles.

3.3 Reproduction and Sources

From the repository root, run the simulator and the checks with Python 3.9 or newer. The simulator needs no third-party packages.

```
cd Assignment1/Tomasulo
python3 tomasulo-python/code/main.py
python3 -m unittest discover -s tests -v
python3 run_benchmarks.py
```

To rebuild the reports, return to the repository root:

```
python3 -m venv .report-venv
.report-venv/bin/pip install -r requirements-report.txt
.report-venv/bin/python scripts/build_assignment_reports.py
```

The builder reruns all 36 cases and checks the saved traces and summary counts. Results must be regenerated before rebuilding a report if the simulator changes.

Sources and limitations

The supplied assignment handout defines the ISA, widths and timing rules. Sample_Tomasulo_Assignment_Report.pdf supplies the report format and the two sample test streams. The Python starter credits Qing, May 2017. Initial Version excerpts come from that starter; My Version excerpts come from the submitted source.

Predictor updates at EX are not rolled back after a later squash. Memory replay can be conservative, and forwarding occupies the memory port for one cycle. Invalid accesses are reported at retirement. Retained instruction history grows with execution length. The arithmetic and hardware limitations are listed in Section 1.

Codex assisted with implementation, testing and report preparation. All numerical tables were checked against the included code and saved outputs.

Ali Alfageeh | October 15, 2026.
GitHub: fqeh/CompArchAssignment1 / Assignment1/Tomasulo